

Optimización Inteligente de Estanterías con IA

Sistema de optimización de beneficios para supermercados
mediante Machine Learning y RAG

+67.794 EUR/mes | +16,2% beneficio

Objetivo del Proyecto



Generar datos de ventas mensuales realistas con LLM para un catalogo de supermercado español (~4.500 productos)



Entrenar y comparar 4 modelos ML (MLP, LSTM, Transformer, PPO) para predecir el impacto de mover de mover productos entre estantes



Construir un pipeline RAG que usa datos históricos de ventas para prever demanda futura



Optimizar la disposición de estantes para cualquier mes futuro usando la mejor combinación de modelos de modelos



Cuantificar el impacto de negocio: medir el beneficio extra vs la disposición actual

Restricciones Iniciales

Restricciones Físicas

- 300 cm de ancho por estante
- 7 niveles de estante por rack
- Estantes a nivel de ojos (3-4) venden mas
- Nivel suelo (1) y superior (7) venden menos
- Productos tienen anchos variables

Datos y Alcance

- Catalogo real: products_macro.csv
- ~4.500 productos en 149 categorías
- Supermercado español
- Estacionalidad realista (ej.: helado en verano)
- Pipeline reproducible end-to-end

Arquitectura del Pipeline

FASE DE ENTRENAMIENTO

01

Generar ventas
mensuales (x12)



02

Entrenar
4 modelos



03

Evaluar y
comparar

FASE DE PRODUCCION

04

Ingestar datos
+ entrenar modelos



05

RAG + LLM
forecast



Ensemble
optimización

Salida: CSV optimizado + informe de beneficios + forecast JSON

Generación de Datos

1 Muestreo de productos (60-90% del catalogo por mes)

2 Estimación LLM (ventas, margen, ancho por lote de 30)

3 Factores estacionales (ej.: helado x1.8 en julio)

4 Asignación a rack y estante (1 rack por categoría)

5 Restricción de ancho: máx. 300cm por estante

6 Capas Anti-Alucinación

- ✓ Prompt restringido
- ✓ Clampeo de valores
- ✓ Validación de longitud
- ✓ Extracción JSON regex
- ✓ Retry + fallback de modelo
- ✓ Heurísticas locales

Factores Estacionales

Mes	Categoría	Multiplicador	Razón
Julio	Helado	x1.8	Calor veraniego
Julio	Agua	x1.6	Hidratación
Diciembre	Chocolate	x1.8	Regalos navideños
Diciembre	Turrón	x2.5	Dulce típico navidad
Enero	Fruta	x0.8	Fuera de temporada

Resultado: 12 ficheros CSV mensuales, cada uno con ~3.000-3.500 productos.
42.301 registros producto-mes en total. Semilla determinista para reproducibilidad.

Datos de Entrenamiento: Simulador Físico

Los CSV son observaciones estáticas, no experimentos. Necesitamos datos causa-efecto: que pasa si movemos un producto de estante.

Formula de Beneficio

$\text{profit} = \text{precio} \times (\text{margen}/100) \times \text{ventas} \times \text{multiplicador_estante}$

Estante 1 (suelo): $\times 0.60$

Estante 2: $\times 0.80$

Estante 3: $\times 0.95$

Estante 4 (ojos): $\times 1.15$

Estante 5: $\times 1.00$

Estante 6: $\times 0.75$

Estante 7 (alto): $\times 0.50$

Ejemplo: Aceite de Oliva

Precio: 8,50 EUR | Margen: 40% | Ventas: 200/mes

En estante 2:

$8,50 \times 0,40 \times 200 \times 0,80 = 544 \text{ EUR}$

En estante 4:

$8,50 \times 0,40 \times 200 \times 1,15 = 782 \text{ EUR}$

profit_lift = +238 EUR

Entrenamiento: 126.903 muestras

Test: 42.301 muestras

Por que un Simulador Físico?

El Problema

Los CSV mensuales son fotos fijas: nos dicen que "el Aceite de Oliva esta en el estante 3 y vende 200 Uds./mes".

Pero NO nos dicen lo mas importante:

Que pasaría si lo movemos al estante 5?

La Solución

retail_physics.py codifica conocimiento de retail (como la posición del estante afecta las ventas) en una formula y genera millones de ejemplos simulados de tipo:

"Mover Aceite de estante 3 a 4: +238 EUR"

"Mover Sal de estante 4 a 1: -9,90 EUR"

"Mover Leche de estante 2 a 6: -45 EUR"

Por que no datos reales?

Ningún supermercado intercambia miles de productos cada día solo para registrar que pasa. Estos datos causa-efecto no existen en el mundo real. Hay que simularlos.

Que aprenden los modelos

Input (X):

Features del producto + estante actual + estante propuesto + contexto del estante

Label (Y):

Cambio exacto en beneficio del rack tras el intercambio

4 Arquitecturas de Modelos



MLP

Feedforward

~53K params

*Puede un mapeo directo
features -> salida resolver
esto?*



LSTM

Secuencial

~45K params

*Importan los efectos de
orden de productos?*



Transformer

Atención

~400K params

*Importan las interacciones
entre productos?*



PPO

Refuerzo (RL)

~35K params

*Puede el ensayo-error
superar al aprendizaje
supervisado?*

MLP: Multi-Layer Perceptron

Arquitectura

Input	10 features
Capa 1	Linear(10,256) + ReLU + Dropout(0.15)
Capa 2	Linear(256,128) + ReLU + Dropout(0.15)
Capa 3	Linear(128,64) + ReLU + Dropout(0.15)
Output	Linear(64,1) -> profit_lift predicho

Características Clave

Mejor optimizador: +1.648 EUR profit lift

Rápido: microsegundos por evaluación

Evalúa cada producto independientemente

7 llamadas por producto (una por estante posible)

Decisiones audaces y directas

Limitación: sin conciencia inter-producto

MSE: 553

80 épocas

~53K parámetros

Transformer: Atención entre Productos

Arquitectura (v3 final)

BatchNorm1d sobre 10 features de entrada

Linear(10, 128) -> proyección a $d_{\text{model}}=128$

4 capas TransformerEncoder (Pre-LN, 4 heads)

Activación GELU, dropout 0.15

Head: Linear(128,64) + ReLU + Linear(64,1)

Evolución del Diseño

v1: Post-LN, MSE 1.021

Entrenamiento inestable

v2: LayerNorm, MSE 823

Mejor, convergencia lenta

v3: BatchNorm+Pre-LN+GELU, MSE 299

Estable y rápido

Fortalezas

Mejor precisión de predicción (MSE: 299)

Contexto bidireccional completo

Computación paralela (sin cuello secuencial)

Mejor score en el ensemble

Limitaciones

Modelo mas grande (~400K params, ~3.6 MB)

Optimizador conservador (+651 EUR)

Requiere mas datos y épocas (150)

Necesita tuning cuidadoso (LR, norm, clipping)

LSTM y PPO

LSTM (Long Short-Term Memory)

Red recurrente: procesa productos secuencialmente manteniendo memoria de ítems anteriores

2 capas apiladas, hidden=64

Entrada: secuencias de 10 productos (batch, 10, 10)

Salida: 10 predicciones de profit_lift

~45K parámetros | 80 épocas | LR: 5e-4

MSE: 699 | Profit Lift: +390 EUR

Limitación: procesamiento unidireccional, solo ve productos anteriores en la secuencia.

PPO (Proximal Policy Optimization)

Aprendizaje por refuerzo: aprende por ensayo y error, sin etiquetas

Actor (política) + Crítico (valor)

Acciones: intercambiar 2 productos de estante

500 episodios de 50 swaps cada uno

~35K parámetros | Entrenado en 1 rack

Profit Lift: +230 EUR

Rendimiento inferior: las recompensas son inmediatas y calculables directamente; RL brilla con recompensas complejas y diferidas.

Métricas de Evaluación: MSE y Profit Lift

MSE (Mean Squared Error)

Que mide:

Precisión con la que el modelo predice profit_lift en EUR. Menor MSE = predicciones mas precisas.

Como se calcula:

$MSE = (1/N) \times \sum (\text{predicción} - \text{real})^2$ sobre 42.301 muestras de test no vistas en entrenamiento.

Como interpretarlo:

MSE 553 (MLP): error medio raíz(553) = ~23,5 EUR/producto

MSE 299 (Transformer): raíz(299) = ~17,3 EUR/producto

Profit Lift (Beneficio Extra)

Que mide:

Beneficio real extra (EUR) al optimizar estantes. Métrica directa de valor de negocio.

Como se calcula:

1) Rack real de test (Rack 2, 40 productos). 2) Beneficio original. 3) Modelo prueba 7 estantes/producto, elige el mejor. 4) Beneficio nuevo. 5) Lift = nuevo - original.

Como interpretarlo:

MLP: +1.648 EUR vs original

Transformer: +651 EUR | Greedy: -2.477 EUR

Por que necesitamos AMBAS

Precisión de predicción $\neq$ calidad de decisión.

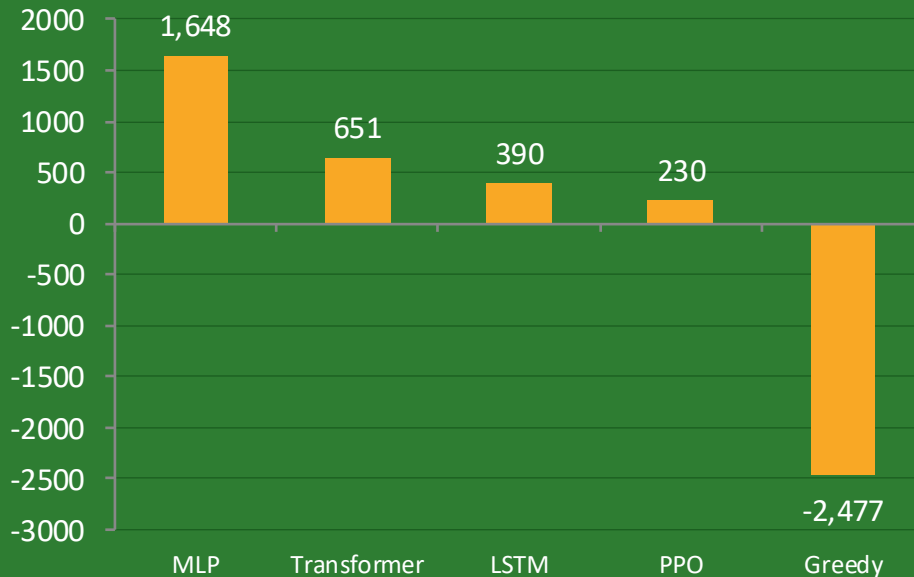
Transformer predice con mitad de error que MLP (299 vs 553), pero MLP genera 2,5x mas beneficio (+1.648 vs +651 EUR). Usar solo MSE nos habría llevado a elegir el modelo equivocado. Profit Lift mide lo que importa: cuanto dinero extra genera el sistema.

Por que no Accuracy / F1 / AUC?

Son métricas de clasificación (predecir categorías). Nuestro problema es regresión: predecir un valor continuo en euros. MSE es la métrica estándar para regresión.

Comparación de Modelos

Profit Lift por Modelo



Precisión (MSE)

Transformer	299
MLP	553
LSTM	699

Paradoja: MSE bajo (Transformer) $\neq$ mejor optimización (MLP). Precisión $\neq$ calidad de decisión.

Greedy baseline (margen alto a nivel de ojos) pierde dinero: -2.477 EUR. Ignora hacinamiento y dependencias.

Ensemble: MLP Propone, Transformer Valida

MLP: Explorador Creativo

Genera 5 layouts candidatos por rack con niveles de ruido crecientes.

Candidato 0: Ruido 0 (greedy puro)

Candidato 1: Ruido 5

Candidato 2: Ruido 10

Candidato 3: Ruido 15

Candidato 4: Ruido 20 (máximo)

Transformer: Juez Preciso

Evalúa cada candidato completo con self-attention. Suma profit_lift predicho de todos los productos. El candidato con mayor puntuación gana.

Su precisión superior (MSE 299) permite distinguir entre layouts "buenos" y "ligeramente mejores".

Por que funciona: Generate-and-Rank

El MLP sabe que estantes son generalmente buenos y genera layouts "aproximadamente correctos" pero diversos. El Transformer, con su precisión superior, elige el ganador. Patrón común en ML: beam search en NLP, evaluador de trayectorias en robótica, evaluador de movimientos en juegos.

Knowledge Base: Ingestión en ChromaDB

Proceso de Ingestión (04_ingest.py)

Dos hilos en paralelo:

Hilo 1: Embeddings (CPU + API)

1. Parsear cada CSV mensual por categoría
2. Generar resúmenes en lenguaje natural por categoría (ej.: nombre, ventas medias, precio medio, estantes mas usados)
3. Embeber con sentence-transformers
4. Almacenar vectores en ChromaDB

Hilo 2: Entrenamiento (CPU + GPU)

1. Generar muestras de entrenamiento con retail_physics
2. Entrenar MLP (80 épocas)
3. Entrenar Transformer (120 épocas)

Que se almacena en ChromaDB

Resúmenes por categoría y mes en lenguaje natural. Cada documento contiene las métricas agregadas de una categoría en un mes concreto:

"Yogures griegos (Enero 2025): ventas medias 180 uds, precio medio 2,40 EUR, estantes mas frecuentes 3-4, 12 productos"

149 categorías x 12 meses = ~1.788 documentos embebidos

Estrategia de Retrieval

Para predecir Enero 2026, se recuperan 3 contextos por categoría:

Enero 2025

Mismo mes año anterior (patrón estacional)

Nov + Dic 2025

Dos meses recientes (tendencia actual)

Pipeline de Predicción

1	RAG Retrieval	ChromaDB: contexto histórico (mismo mes año anterior + 2 meses recientes)
2	Cargar datos base	CSV mas reciente como línea base (ej.: diciembre 2025, 3.133 productos)
3	LLM Forecast	LLM predice multiplicadores de ventas por categoría usando contexto RAG
4	Aplicar forecast	Multiplicar ventas base por multiplicadores del LLM por categoría
5	Ensemble	MLP genera 5 layouts, Transformer puntúa, gana el mejor
6	Guardar resultados	CSV optimizado + JSON forecast + tabla comparativa de beneficios
Los datos RAG nunca llegan a MLP/Transformer directamente. Fluyen vía LLM, que los convierte en ajustes numéricos de ventas.		

Decisiones de Diseño Clave

RAG + LLM vs series temporales

Solo 12 meses de datos; el LLM aporta conocimiento del mundo real que ARIMA/Prophet no pueden con tan pocos datos

Ruido gaussiano en candidatos

Simple, efectivo, crea gradiente de diversidad natural. Alternativas (beam search, temperature sampling) mas complejas sin beneficio claro

Pre-LN Transformer (no Post-LN)

Gradientes mas estables, convergencia mas rápida; estándar moderno desde GPT-2

LLM con temperatura 0.3

Estimaciones consistentes y factuales, con ligera variación para evitar outputs degenerados

BatchNorm en inputs del Transformer

Features con escalas muy diferentes (precio 0,50-50 vs ventas 1-500); redujo MSE de 823 a 299

Ingestión paralela

Embeddings (CPU+API) y entrenamiento (CPU+GPU) usan recursos diferentes; reduce tiempo de 18 a 10 min

Resultados Finales

+67.794 EUR

Beneficio mensual extra

+16,2%

Incremento de beneficio

3.133

Productos optimizados

Métrica	Valor
Racks cubiertos	149
Categorías	149
Mejor modelo predicción	Transformer (MSE: 299)
Mejor modelo optimización	MLP (Profit Lift: + 1.648 EUR)
Enfoque final	Ensemble MLP + Transformer
Pipeline	RAG + LLM forecast + Ensemble



+67.794 EUR/mes

+16,2% de beneficio con optimización inteligente de estanterías

MLP propone | Transformer valida | LLM predice | RAG contextualiza

Optimización de Estanterías con IA